EDA d’une série temporelle orientée **LightGBM**

L’idée générale : avec **LightGBM**, tu ne donnes pas “une série temporelle” au modèle, tu lui donnes **un tableau de features**.

Donc une bonne EDA ne sert pas seulement à “comprendre” les données, elle sert surtout à décider :

- **quelles features temporelles créer**
- **comment éviter la fuite de données**
- **comment découper train/validation/test**
- **quelles variables garder ou supprimer**
- **comment représenter la saisonnalité, les lags, les effets ville/région**

---

1. Compréhension du problème métier et de la cible

Ce qu’on vérifie

- Quelle variable veut-on prédire exactement ?
- À quel horizon ?
  - demain (t+1)
  - dans 7 jours (t+7)
  - risque, indice, température, pluie, etc.
- Est-ce une **régression** ou une **classification** ?
- Quel est le niveau de prédiction ?
  - par ville
  - national

  - par région

- Quelle est l’utilité métier de la cible ?

Ce que ça implique pour LightGBM

- LightGBM a besoin d’une **cible clairement définie**

- Le choix de l’horizon change totalement les features :

  - pour t+1, les lags courts sont très utiles

  - pour t+7, il faut davantage de rolling, cumuls et saisonnalité

- Si la cible est mal définie, le modèle peut être bon techniquement mais inutile en pratique

- Il faut savoir dès le début si tu construis :

  - **un seul modèle global** pour toutes les villes

  - ou **un modèle par ville**

---

2. Compréhension de la structure du dataset

Ce qu’on vérifie

- Nombre de lignes et colonnes

- Types de variables

- Variables numériques, catégorielles, temporelles

- Présence d’identifiants (`id`)

- Présence d’entités multiples (`city`, `region`)

- Période couverte

- Fréquence des données

Ce que ça implique pour LightGBM

- LightGBM travaille sur des **colonnes tabulaires**

- Les colonnes comme `id` ne doivent en général pas être utilisées

- `city` et `region` peuvent être très utiles comme **features catégorielles**

- `latitude` et `longitude` sont très utiles car elles permettent au modèle d'apprendre des différences climatiques spatiales

- Avec plusieurs villes, toutes les opérations temporelles doivent être faites **par ville**

- Si tu mélanges les villes pour créer les lags, tu crées des features fausses

---

3. Vérification de la dimension temporelle

Ce qu'on vérifie

- La colonne date est-elle bien au bon format ?

- Les données sont-elles triées chronologiquement ?

- La fréquence est-elle bien journalière ?

- Y a-t-il des dates manquantes ?

- Y a-t-il des doublons date-ville ?

- Chaque ville a-t-elle une série continue ?

Ce que ça implique pour LightGBM

- Si la date n'est pas propre, les lags et rolling seront faux

- Si les lignes ne sont pas triées, `lag_1` ne représentera pas "hier"

- Si des jours manquent :

  - lag_1 peut représenter en réalité il y a 3 jours

  - rolling 7 jours devient incohérent
- Si tu as des doublons date-ville :
  - une même journée apparaît plusieurs fois
  - les agrégations et features seront biaisées
- Donc avant tout feature engineering, il faut garantir :
  - **une ligne = une ville à une date donnée**

---

4. Vérification de la complétude par entité

Ce qu'on vérifie

- Est-ce que toutes les villes ont la même période ?
- Certaines villes commencent-elles plus tard ?
- Certaines villes ont-elles plus de trous que d'autres ?
- Certaines variables manquent pour certaines régions ou certaines périodes ?

Ce que ça implique pour LightGBM

- Si certaines villes ont peu d'historique, elles auront moins de lags valides
- Cela peut introduire un biais :
  - le modèle apprend surtout sur les villes les plus complètes
- Si une variable est manquante sur beaucoup de villes/périodes :
  - soit on l'impute intelligemment
  - soit on l'exclut
- Si une ville est trop incomplète :
  - elle peut dégrader le modèle global

- Cette étape aide à décider si tu fais :
  - un modèle global
  - ou un sous-ensemble de villes bien renseignées

---

5. Nettoyage des données

Ce qu'on vérifie

- Doublons exacts
- Doublons sur `(date, city)`
- Valeurs manquantes
- Valeurs incohérentes
- Valeurs impossibles physiquement
- Colonnes constantes ou quasi constantes

Ce que ça implique pour LightGBM

- LightGBM supporte relativement bien certaines imperfections, mais :
  - des duplicats biaisent l'apprentissage
  - des erreurs physiques ajoutent du bruit
- Les colonnes constantes n'apportent aucune information
- Les NaN :
  - LightGBM peut les gérer, mais il vaut mieux comprendre **pourquoi** ils existent
- Les incohérences physiques doivent être corrigées ou documentées
- Si une variable est toujours 0 (`snowfall_sum` dans ton cas), elle doit être supprimée car elle n'aide pas

---

6. Analyse des valeurs manquantes

Ce qu'on vérifie

- Combien de valeurs manquantes par variable
- Répartition temporelle des manquants
- Répartition géographique des manquants
- Sont-ils aléatoires ou structurés ?
- Y a-t-il des blocs entiers manquants ?

Ce que ça implique pour LightGBM

- Même si LightGBM tolère les NaN, leur structure est importante :
  - si les NaN sont aléatoires → impact faible
  - s'ils sont concentrés sur une saison ou une région → risque de biais
- Une longue période manquante n'est pas un simple "trou", c'est une **rupture d'information**
- Si on remplit mal, on injecte un faux signal
- Pour les séries temporelles, l'imputation doit respecter le temps :
  - interpolation temporelle
  - forward fill avec prudence
- Il ne faut jamais utiliser une méthode qui exploite le futur si on veut simuler la réalité

---

7. Statistiques descriptives

Ce qu’on vérifie

- Moyenne, médiane, écart-type
- Min et max
- Quantiles
- Dispersion
- Asymétrie
- Présence de valeurs extrêmes

Ce que ça implique pour LightGBM

- LightGBM n’a pas besoin de normalisation stricte comme LSTM, mais :
  - les statistiques aident à détecter les anomalies
  - elles indiquent quelles variables sont très volatiles
- Une variable très dispersée peut contenir beaucoup d’information… ou beaucoup de bruit
- Une variable trop stable risque d’être peu utile
- Cette étape aide à repérer :
  - les colonnes suspectes
  - les colonnes à transformer
  - les colonnes candidates à des interactions métier

---

8. Visualisation temporelle

Ce qu’on vérifie

- Tendance long terme
- Variations saisonnières
- Pics et creux
- Changements de régime
- Différences entre villes/régions
- Effets extrêmes

Ce que ça implique pour LightGBM

- LightGBM ne “voit” pas le temps naturellement
- Tout ce que tu observes ici doit ensuite être traduit en **features**
- Si tu vois une saisonnalité annuelle :
  - il faudra créer `month`, `day_of_year`, `sin/cos`
- Si tu vois des effets courts :
  - il faudra créer des lags courts et rolling courts
- Si tu vois des différences fortes entre villes :
  - `city`, `region`, `latitude`, `longitude` deviennent essentielles
- Si tu vois des ruptures :
  - il peut falloir des features de période ou de régime

---

9. Analyse de la saisonnalité

Ce qu’on vérifie

- Cycle annuel
- Cycle mensuel

- Cycle hebdomadaire
- Intensité de la saisonnalité
- Différences de saisonnalité selon la ville

Ce que ça implique pour LightGBM

- Il faut encoder la saisonnalité explicitement
- Les variables temporelles simples :
  - mois
  - trimestre
  - saison
  - jour de l'année
- Les variables cycliques :
  - `sin` et `cos` du mois
  - `sin` et `cos` du jour de l'année
- Si la saisonnalité varie selon la région :
  - LightGBM peut apprendre l'interaction entre saison et ville/région
- Si tu n'encode pas la saisonnalité, LightGBM perd une énorme partie du signal

---

10. Analyse de tendance et non-stationnarité

Ce qu'on vérifie

- La série monte-t-elle ou descend-elle dans le temps ?
- Les niveaux changent-ils selon les années ?
- Les variances changent-elles ?

- Y a-t-il des ruptures structurelles ?

Ce que ça implique pour LightGBM

- Contrairement à ARIMA, LightGBM n'exige pas la stationnarité

- Mais si la tendance existe, il faut la représenter :

  - `year`

  - index temporel

  - features calendaires

- Si certaines variables changent structurellement avec le temps, le modèle doit pouvoir l'apprendre

- Si la tendance est forte et non représentée, LightGBM essaiera de l'approximer de façon indirecte, souvent moins bien

---

11. Analyse des outliers

Ce qu'on vérifie

- Valeurs extrêmes

- Fréquence des valeurs extrêmes

- Temporalité des extrêmes

- Cohérence physique des extrêmes

- Concentration géographique des extrêmes

Ce que ça implique pour LightGBM

- Les arbres sont plutôt robustes aux outliers

- Donc on ne supprime pas automatiquement les extrêmes

- En météo, beaucoup d'outliers sont en fait des **événements réels**

- Il faut distinguer :

  - erreur de mesure

  - événement climatique rare mais réel

- Si on supprime tous les extrêmes, on supprime justement les épisodes les plus intéressants pour la résilience climatique

- Il faut donc garder les vrais extrêmes, corriger seulement les faux

---

12. Corrélations et redondances entre variables

Ce qu'on vérifie

- Corrélations entre variables météo

- Variables presque identiques

- Variables fortement redondantes

- Relations avec la cible

Ce que ça implique pour LightGBM

- LightGBM gère mieux la multicolinéarité qu'une régression linéaire

- Donc les corrélations fortes ne sont pas forcément un problème

- Mais une forte redondance peut :

  - alourdir inutilement le modèle

  - rendre l'interprétation plus difficile

- Cela aide à choisir quelles variables garder si plusieurs racontent la même chose

- Cette étape aide aussi à imaginer des **features métier** utiles :

- amplitude thermique = max - min
- ratio sunshine/daylight
- balance hydrique = pluie - évapotranspiration

---

13. Analyse de la relation cible-features

Ce qu'on vérifie

- Quelles variables semblent liées à la cible ?
- La relation est-elle linéaire ou non-linéaire ?
- Les effets changent-ils selon la saison ou la région ?
- Les lags de la cible sont-ils très prédictifs ?

Ce que ça implique pour LightGBM

- LightGBM est très bon pour les relations non-linéaires
- Donc même si la corrélation linéaire est faible, la variable peut être utile
- Cette étape permet d'identifier :
  - les variables fortes
  - les variables faibles
  - les interactions potentielles
- Très souvent, les **lags de la cible** seront parmi les meilleures variables
- Si la cible dépend de plusieurs facteurs combinés, LightGBM est un excellent choix

---

14. Analyse d’autocorrélation

Ce qu’on vérifie

- Dépendance à court terme
- Dépendance hebdomadaire
- Dépendance mensuelle
- Dépendance saisonnière
- Quels retards semblent importants

Ce que ça implique pour LightGBM

- Cette étape sert directement à choisir les **lags**
- Si forte autocorrélation à 1 jour :
  - créer `lag_1`
- Si autocorrélation à 7 jours :
  - créer `lag_7`
- Si effet à 30 jours :
  - créer `lag_30`
- Cela évite de créer des dizaines de lags inutiles
- L’idée n’est pas de tout créer, mais de créer les lags qui capturent la mémoire réelle de la série

---

15. Analyse par ville / région

Ce qu’on vérifie

- Les villes ont-elles des comportements très différents ?

- Certaines régions ont-elles des climats proches ?
- La saisonnalité est-elle uniforme ?
- La cible a-t-elle la même dynamique partout ?

Ce que ça implique pour LightGBM

- Si les villes diffèrent fortement, il faut intégrer :
  - `city`
  - `region`
  - `latitude`
  - `longitude`
- Si le comportement est très hétérogène, un modèle global peut avoir du mal
- Mais LightGBM peut apprendre des interactions entre :
  - ville × saison
  - région × pluie
  - latitude × radiation
- Donc un **modèle global enrichi spatialement** est souvent une très bonne stratégie
- Cette étape aide à choisir entre :
  - modèle global
  - modèle par ville
  - modèle par région

---

16. Feature engineering temporel

Ce qu’on crée

- Année

- Mois

- Jour

- Jour de semaine

- Jour de l’année

- Semaine de l’année

- Saison

- Encodages cycliques (`sin/cos`)

Ce que ça implique pour LightGBM

- Comme LightGBM n’a pas de mémoire temporelle native, ces features sont cruciales

- Le mois codé 1 à 12 peut suffire parfois, mais le cyclique est souvent meilleur

- Les calendaires capturent la saisonnalité globale

- Elles permettent au modèle d’anticiper les régimes climatiques même sans lag

---

17. Création des lags

Ce qu’on crée

- Lags de la cible

- Lags des variables météo importantes

- Lags courts : 1, 2, 3 jours

- Lags moyens : 7, 14 jours

- Lags longs : 30 jours

Ce qu’on vérifie

- Les lags sont bien créés **par ville**
- Ils ne mélangent pas les entités
- Ils respectent l’ordre temporel

Ce que ça implique pour LightGBM

- Les lags sont souvent les features les plus puissantes
- Ce sont eux qui donnent au modèle la mémoire du passé
- Si mal créés, tout le modèle devient faux
- Ils créent des NaN au début des séries
- Plus tu crées des lags longs, plus tu perds de lignes exploitables

---

18. Création des rolling features

Ce qu’on crée

- Moyenne glissante
- Médiane glissante
- Min / max glissants
- Écart-type glissant
- Cumul glissant de pluie
- Cumul de chaleur ou d’évapotranspiration

Ce qu’on vérifie

- Fenêtres cohérentes :

  - 3 jours
  - 7 jours
  - 14 jours
  - 30 jours
- Pas de fuite de données
- Calcul par ville

Ce que ça implique pour LightGBM

- Les rolling résument le contexte récent
- Très utiles pour capturer :
  - tendance locale
  - instabilité
  - stress cumulé
- Pour la météo, les cumuls sont souvent plus informatifs que la valeur brute d'un seul jour
- Ex. : pluie cumulée sur 7 jours > pluie du jour

---

19. Création de variables métier

Ce qu'on vérifie / construit

- Indices composites pertinents
- Exemples :
  - amplitude thermique
  - balance hydrique
  - rapport sunshine/daylight

- stress thermique
- indice de sécheresse
- VRI ou autre indicateur de risque

Ce que ça implique pour LightGBM

- C’est ici que tu injectes de l’intelligence métier
- LightGBM adore ce type de features car elles condensent plusieurs signaux utiles
- Ces variables peuvent faire la différence au hackathon
- Il faut qu’elles soient cohérentes et justifiables
- Elles doivent être calculables avec les informations réellement disponibles au moment de la prédiction

---

20. Vérification de la disponibilité future des features

Ce qu’on vérifie

- Est-ce que chaque feature sera connue au moment où on veut prédire ?
- Une variable météo brute de demain est-elle disponible ou non ?
- Les rolling utilisent-ils uniquement le passé ?
- Les lags utilisent-ils uniquement le passé ?

Ce que ça implique pour LightGBM

- C’est la question centrale de la **fuite de données**
- Une feature peut sembler excellente mais être inutilisable en vraie vie
- Il faut distinguer :

  - variables connues à l’avance (date, saison, ville)
  - variables observées seulement jusqu’à aujourd’hui
- Toute feature doit respecter ce cadre réel
- Sinon ton score sera artificiellement élevé

---

21. Création de la cible

Ce qu’on vérifie
- La cible est bien décalée dans le futur
- Pas de confusion entre variable courante et variable à prédire
- La cible est définie par ville
- Les dernières lignes sans cible sont gérées

Ce que ça implique pour LightGBM
- Sans décalage correct, tu fais de la fuite
- Le modèle apprendra à reproduire le présent au lieu de prévoir le futur
- Il faut être extrêmement rigoureux ici
- Cette étape détermine tout le pipeline d’entraînement

---

22. Encodage des variables catégorielles

Ce qu’on vérifie

- Quelles variables sont catégorielles :
  - `city`
  - `region`
  - éventuellement `weather_code`
- Faut-il les encoder ?
- Leur cardinalité est-elle raisonnable ?

Ce que ça implique pour LightGBM

- LightGBM gère très bien les variables catégorielles
- `city` est probablement une variable importante
- `region` aussi
- `weather_code` peut être très informatif
- Il faut juste s'assurer que l'encodage choisi est cohérent
- Les catégorielles peuvent aider le modèle à apprendre des comportements locaux très différents

---

23. Analyse de la cible

Ce qu'on vérifie

- Distribution de la cible
- Équilibre ou déséquilibre si classification
- Variabilité selon ville et saison
- Difficulté intrinsèque de la cible
- Présence de pics extrêmes

Ce que ça implique pour LightGBM

- Si la cible est très asymétrique :
  - peut-être transformer la cible
  - ou choisir une métrique robuste
- Si la cible a beaucoup d'extrêmes :
  - RMSE sera plus sévère
- Si classification déséquilibrée :
  - il faudra gérer les classes
- Si la cible est très stable, un modèle naïf peut déjà être fort
- Cette étape permet de définir une **baseline réaliste**

---

24. Baselines simples

Ce qu'on vérifie

- Prédire la dernière valeur
- Prédire la moyenne glissante
- Prédire la valeur de la semaine précédente

Ce que ça implique pour LightGBM

- Avant de juger LightGBM, il faut avoir une baseline simple
- Si LightGBM ne bat pas un modèle naïf, il y a un problème :
  - fuite
  - mauvais features

  - mauvaise cible
  - split incorrect
- Une baseline est indispensable pour interpréter les résultats

---

25. Split temporel

Ce qu'on vérifie

- Respect absolu de la chronologie
- Train = passé
- Validation = période intermédiaire
- Test = période récente
- Pas de shuffle
- Split cohérent par date

Ce que ça implique pour LightGBM

- Même si LightGBM est un modèle tabulaire, le split doit rester **temporel**
- Si tu fais un split aléatoire, tu triches
- Les performances deviendront artificiellement élevées
- Le split doit idéalement être identique pour toutes les villes
- C'est une étape critique pour avoir une évaluation crédible

---

26. Validation croisée adaptée

Ce qu’on vérifie

- Utilisation d’une validation temporelle
- Fenêtres glissantes ou expansives
- Cohérence entre folds

Ce que ça implique pour LightGBM

- Très utile pour tuner LightGBM proprement
- Évite de choisir des hyperparamètres sur un seul split chanceux
- Permet d’estimer la stabilité du modèle dans le temps
- Beaucoup plus fiable qu’un simple train/test

---

27. Préparation finale du tableau de features

Ce qu’on vérifie

- Quelles colonnes garder
- Quelles colonnes supprimer
- Présence de NaN résiduels
- Features numériques / catégorielles bien identifiées
- Absence de colonnes interdites :
  - date brute non transformée si inutile
  - id
  - cible future
  - colonnes fuyardes

Ce que ça implique pour LightGBM

- C’est le dataset final d’entraînement
- Il doit être propre, cohérent, sans ambiguïté
- La qualité de cette table détermine la qualité du modèle
- À ce stade, LightGBM doit recevoir un vrai tableau de décision

---

28. Ce qu’une bonne EDA doit produire spécifiquement pour LightGBM

À la fin de l’EDA, tu dois avoir :

1. Une compréhension claire de la cible

- Quoi prédire
- À quel horizon
- Pourquoi

2. Un diagnostic temporel propre

- Dates propres
- Fréquence régulière
- Pas de doublons
- Séries cohérentes par ville

3. Une stratégie d’imputation

- Justifiée

- Temporelle

- Sans fuite

4. Une stratégie de features

- calendaires

- lags

- rolling

- cumuls

- métier

- spatiales

5. Une stratégie de modélisation

- modèle global ou par ville

- variables catégorielles intégrées

- validation temporelle

6. Une stratégie anti-fuite

- aucune feature du futur

- rolling et lags bien construits

- split temporel correct

---

29. Ce que ton EDA doit conclure si tu veux orienter le projet vers LightGBM

Tu dois arriver à ce genre de conclusion :

> Les données sont journalières, multivariées et multi-villes.

> La série présente une forte structure temporelle avec saisonnalité, dépendances de court terme et différences géographiques marquées.

> Comme LightGBM ne modélise pas explicitement le temps, il est nécessaire de transformer cette structure en variables explicites : lags, statistiques glissantes, variables calendaires, informations spatiales et indicateurs métier.

> L'EDA montre également la nécessité d'un split strictement chronologique et d'une construction des features par ville pour éviter toute fuite de données.

> Ce cadre rend LightGBM particulièrement adapté, car il peut exploiter simultanément les informations historiques, saisonnières, géographiques et météorologiques dans un cadre tabulaire robuste et performant.

---

30. Résumé ultra simple

Pour une EDA orientée **LightGBM**, tu dois répondre à 5 grandes questions :

1. Les données temporelles sont-elles propres ?

- dates
- ordre
- fréquence
- continuité

2. Quelle information du passé aide à prédire le futur ?

- lags
- rolling

- cumuls

- autocorrélation

3. Quels effets récurrents existent ?

- saisonnalité

- calendrier

- différences ville/région

4. Quelles variables sont réellement utiles ?

- météo brute

- variables métier

- variables spatiales

- catégorielles

5. Comment éviter la fuite ?

- features construites avec le passé seulement

- cible bien décalée

- split chronologique strict

Si tu veux, je peux maintenant te faire la **version rapport/soutenance**, c'est-à-dire :

"EDA en séries temporelles orientée LightGBM — texte propre à mettre dans le rapport"

avec un style académique, propre et directement réutilisable.